

编译原理实验 PR002

组员：谢翊凡，闫涛，曹建文

Part1: 符号表的构建

在第一部分，我们首先完成了对符号表的构造，用于记录和查询变量、函数的作用域、类型等信息。结构大致如下：

```
class SymbolTable {
public:
    std::unordered_map<std::string, SymEntry> symentries; // 当前作用域的符号
    std::unordered_map<std::string, FunEntry> funentries; // 当前作用域的函数
    SymbolTable* parent; // 指向父作用域的符号表
    int scopeLevel; // 当前作用域层级
```

Figure 1: SymbolTable 类的基本结构

我们以如下设计进行了初始化，使得作用域层级从零开始随深度递增：

```
SymbolTable::SymbolTable() : parent(nullptr), scopeLevel(0) {}
```

Figure 2: SymbolTable 构造函数初始化作用域层级

通过下面的代码，实现作用域的进入，创建一个新符号表，并将当前表设为 parent，同时作用域层级加一，每当作用域嵌套以上述函数实现，即表示进入了更深一层的作用域：

```
// 进入新作用域（如函数体、代码块）
SymbolTable* SymbolTable::enterScope() {
    SymbolTable* newScope = new SymbolTable();
    newScope->parent = this;
    newScope->scopeLevel = scopeLevel + 1;
    return newScope;
}
```

Figure 3: SymbolTable::enterScope() 函数实现

后续，我们对符号和函数在符号表中的插入做了处理，二者思路中相同的部分是都查找是否有重名情况出现，若有则报错处理；若无异常，则插入到符号表中；

```

// 插入符号 (若存在同名符号则报错)
void SymbolTable::insertsym(const std::string& name, const SymType& type) {
    if (symentries.find(name) != symentries.end()) {
        throw std::runtime_error("重复定义变量: " + name);
    }
    symentries[name] = {name, type, scopeLevel};
}

// 插入函数 (若存在同名符号则报错)
void SymbolTable::insertfun(const std::string& name, const FunType& type,
                           const std::vector<std::string>& params_name,
                           const std::vector<SymType>& params_type) {
    if (funentries.find(name) != funentries.end()) {
        throw std::runtime_error("重复定义函数: " + name);
    }
    funentries[name] = {name, type, params_name, params_type, scopeLevel};
}

```

Figure 4: 符号和函数名字的存储处理逻辑

关于查找符号和查找函数，思路有所不同：查找符号时，沿每一层级逐层向上，递归查找；而查找函数时，我们一路查找到最顶层，即单层作用域中查找。具体实现如下：

```

// 查找符号 (递归查找父作用域)
const SymEntry* SymbolTable::lookupsym(const std::string& name) const {
    if (auto it = symentries.find(name); it != symentries.end()) {
        return &it->second;
    }
    return parent ? parent->lookupsym(name) : nullptr;
}

// 查找函数
const FunEntry* SymbolTable::lookupfun(const std::string& name) const {
    const SymbolTable* global = this;
    while (global->parent != nullptr) { // 循环找到最顶层
        global = global->parent;
    }
    if (auto it = global->funentries.find(name); it != global->funentries.end()) {
        return &it->second;
    }
    return nullptr;
}

```

Figure 5: 符号表符号与函数查找逻辑实现

除此之外我们实现了一些辅助函数，例如类型比较和字符串类型映射。类型比较重，比较变量是否为相同基本类型，之后检查是否数组，进而比较维度等。若比较均通过，则表明二者为同一类型；后者则是主要分别实现了枚举到字符串以及字符串到枚举之间的互相映射；在上述实现中，我们用到的 `symtype` 等结构体在 `symboltable.h` 中进行了定义。

Part2: LLVM IR 中间代码的生成

符号表构建完成后，我们在实验的第二部分里实现了中间代码的具体生成，结合了语法树的访问和 LLVM IR 中间代码的生成等。下面将结合具体代码分析实现过程：

```
4
5  Analysis::Analysis() :
6      irGen(std::make_unique<IRGenerator>()),
7      globalTable(new SymbolTable()),
8      current(globalTable),
9      Bool({"bool", false, {}, false}),
10     inLoop(0),
11     return_exist(0)
12 {
13     initBuiltinFunctions();
14 }
15
16 Analysis::~~Analysis() {
17     delete globalTable;
18 }
19
```

Figure 6: LLVM IR 中间代码生成：CompUnit 定义与初始处理

```
void Analysis::initBuiltinFunctions() {
    auto addBuiltin = [&](const std::string& name, FunType retType,
                        const std::vector<std::string>& params,
                        const std::vector<SymType>& paramTypes) {
        globalTable->insertfun(name, retType, params, paramTypes);

        // 生成IR函数声明
        FunEntry entry{name, retType, params, paramTypes, 0};
        irGen->generateFunctionDef(entry);
    };

    addBuiltin("print_int", FunType::VOID, {"n"}, {"int", false, {}, false});
    addBuiltin("print_float", FunType::VOID, {"f"}, {"float", false, {}, false});
    addBuiltin("print_char", FunType::VOID, {"c"}, {"char", false, {}, false});
    addBuiltin("get_int", FunType::INT, {}, {});
    addBuiltin("get_float", FunType::FLOAT, {}, {});
    addBuiltin("get_char", FunType::CHAR, {}, {});
}
```

Figure 7: LLVM IR 中间代码生成：CompUnit 定义与初始处理

首先是构造与初始化的完成。我们在上面的函数里完成了 LLVM IR 代码生成器 `irgen` 的初始化，设定布尔类型，同时初始化了一些内置函数，插入到符号表和 IR 模块中；在 `initbuiltinfunctions()` 函数里生成对应的 LLVM 函数声明。之后实现了内置函数的初始化。

用下面的函数分别实现了维度索引的有效性检查和顶层语义分析入口的实现：

```

38
39 bool Analysis::checkMultiDimInit(const SymType& arrayType,
40                                CACTParser::ConstInitValContext* ctx,
41                                size_t dimIndex) {
42     // 检查维度索引是否有效

```

Figure 8: 维度索引检查与顶层语义分析入口实现

```

std::any Analysis::visitCompUnit(CACTParser::CompUnitContext *ctx) {

```

Figure 9: 维度索引检查与顶层语义分析入口实现

在后续代码中，我们继续完成了常量与变量的声明分析。具体分析过程大致为：提取变量类型，识别数组维度信息，检查初始化合法性，生成 LLVM 中 globalvariable 和 alloca insert 部分。

关于多维数组的初始化，我们分为了展开和平铺逐层初始化，在 checkMultiDimInit、generateNestedArrayInit、fillMultiDimInit 等函数实现了较完整的数组 IR 构造，包括 LLVM 的 constantarray 部分等，完成了 IR 变量声明环节。

```

553 std::any Analysis::visitVarDecl(CACTParser::VarDeclContext *ctx) {
554     for (auto varDefCtx : ctx->varDef()) {
555         else{
556             } else {
557                 if (isFlatInit) {
558
559                     llvm::Value* elemPtr = builder.CreateInBoundsGEP(
560                         arrayAlloc->getType()->getPointerElementType(),
561                         arrayAlloc,
562                         indices,
563                         "array.elem");
564                     builder.CreateStore(initValues[i], elemPtr);
565                 }
566             } else {
567                 // 多维初始化
568                 if (!checkMultiDimInit(varType, constInitValCtx, 0)) {
569                     std::cerr << "Error at line " << ctx->getStart()->getLine()
570                         << ": Invalid multi-dimensional array initialization" << std::endl;
571                     exit(1);
572                 }
573                 // 创建数组类型
574                 llvm::Type* arrayLLVMType = irGen->getLLVMType(varType);
575             }
576         }
577     }
578 }

```

Figure 10: 多维数组初始化与变量 IR 生成流程

```

834 //llvm::errs() << "\n";
835 // 生成IR变量声明
836 llvm::Value* alloca =
837 new llvm::GlobalVariable(
838     /*Module=*/ *irGen->getModule(),
839     /*Type=*/ arrayLLVMType,
840     /*isConstant=*/ false, // 常量数组
841     /*Linkage=*/ llvm::GlobalValue::ExternalLinkage,
842     /*Initializer=*/ init,
843     /*Name=*/ Ident);
844
845 // 在符号表中保存LLVM值
846 if (auto symEntry = current->lookupsym(Ident)) {
847     const_cast<SymEntry*>(symEntry)->type.value = alloca;
848 }
849
850 else{
851     // 局部变量初始化
852     llvm::Value* arrayAlloc = builder.CreateAlloca(arrayLLVMType, nullptr, Ident);
853     // 处理初始化值
854     std::vector<llvm::Value*> indices;
855     fillMultiDimInit(varType, varDefCtx->constInitVal(), arrayAlloc, 0, indices);
856     if (auto symEntry = current->lookupsym(Ident)) {
857         const_cast<SymEntry*>(symEntry)->type.value = arrayAlloc;
858     }
859 }

```

Figure 11: 多维数组初始化与变量 IR 生成流程

关于子语义检查和表达式处理，我们采用了这样的设计：类型检查时在所有的表达式和赋值操作前调用 `isSameType`，防止不一致的类型混用。

```

if (!isSameType(conType, exptype)) {
    std::cerr << "Error at line " << ctx->getStart()->getLine()
    << ": Type mismatch during initialization assignment. Expected "
    << conType.base_type << ", got " << exptype.base_type << std::endl;
    exit(1);
}

```

Figure 12: 类型检查 `isSameType` 逻辑实现

后续，我们通过一系列函数操作处理语法规则下对应地表达式类型，举例如下，是 `visitUnaryExp` 函数处理对应表达式的过程。

```

std::any Analysis::visitUnaryExp(CACTParser::UnaryExpContext *ctx) {
    //std::cout << "visit UnaryExp" << std::endl;
    auto& builder = irGen->getBuilder();

    if (ctx->primaryExp() != nullptr) {
        // 处理基本表达式
        return visitPrimaryExp(ctx->primaryExp());
    }
    else if (ctx->unaryExp() != nullptr) {
        // 处理一元运算符
        auto unaryResult = std::any_cast<std::pair<SymType, llvm::Value*>>(visitUnaryExp(ctx->unaryExp()));
        SymType type = unaryResult.first;
        llvm::Value* value = unaryResult.second;

        if (ctx->getText().find("!") == 0) {
            // 逻辑非运算
            if (!isSameType(type, Bool)) {
                std::cerr << "Error: Logical NOT operator '!' requires boolean operand" << std::endl;
                exit(1);
            }
            return std::make_pair(Bool, builder.CreateNot(value, "logicalnot"));
        }
        else if (ctx->getText().find("-") == 0) {
            // 负号运算
            if (type.base_type == "int") {
                return std::make_pair(type, builder.CreateNeg(value, "neg"));
            }
            else if (type.base_type == "float") {
                return std::make_pair(type, builder.CreateFNeg(value, "fneg"));
            }
        }
        else {
            std::cerr << "Error: Unary '-' operator requires numeric operand" << std::endl;
            exit(1);
        }
    }
}

```

Figure 13: LLVM IR 控制流语句处理流程

关于语句块和控制结构，我们在 `visitStmt` 函数中实现了对 `if`、`while`、`break`、`continue`、`return` 等的处理，也通过如下 `visitBlock()` 函数实现了新作用域创建，当前 LLVM 函数的获取、函数参数和局部变量的处理等：

```

// 语句块与语句规则
std::any Analysis::visitBlock(CACTParser::BlockContext *ctx) {
    // 1. 进入新作用域
    current = current->enterScope();

    auto* func_def = dynamic_cast<CACTParser::FuncDefContext*>(ctx->parent);
    auto* stmtctx = dynamic_cast<CACTParser::StmtContext*>(ctx->parent);

    // 2. 处理函数参数（如果是函数体块）
    llvm::Function* currentFunction = irGen->getFunction(current_funname.top());
    if (func_def != nullptr) {
        std::string funcName = func_def->Ident()->getText();
        current_funname.push(funcName);
        current_rettype.push(stringToFunType(func_def->funcType()->getText()));

        // 获取当前LLVM函数
        currentFunction = irGen->getFunction(funcName);

        // 处理函数参数
        if (func_def->funcFParams()) {
            auto argIter = currentFunction->arg_begin();

            for (auto param : func_def->funcFParams()->funcFParam()) {
                // 解析参数类型
                SymType type;
                type.base_type = param->bType()->getText();
                type.is_const = false;

                std::vector<int> array_dims;
                if (param->getText().find("[ ]") != -1) {
                    array_dims.push_back(-1);
                }
            }
        }
    }
}

```

Figure 14: `visitBlock` 函数实现过程

```
std::any Analysis::visitStmt(CACTParser::StmtContext *ctx) {
    //std::cout << "visit Stmt" << std::endl;
    auto& builder = irGen->getBuilder();
    llvm::Function* currentFunc = builder.GetInsertBlock()->getParent();
```

Figure 15: visitStmt 函数处理过程

我们在 visitFuncDef 函数中，实现了提取与保存函数返回类型和参数列表等工作，后续在 visitUnaryExp 函数中处理函数调用，具体将检查函数参数数量与类型。

```
std::any Analysis::visitFuncDef(CACTParser::FuncDefContext *ctx) {
    //std::cout << "visit FuncDef" << std::endl;

    // 1. 解析函数返回类型
    FunType retType = stringToFunType(ctx->funcType()->getText());
    std::string funcName = ctx->Ident()->getText();

    // 2. 解析函数参数
    std::vector<std::string> paramNames;
    std::vector<SymType> paramTypes;
    std::vector<llvm::Type*> llvmParamTypes;

    if (auto paramsCtx = ctx->funcFParams()) {
        for (auto paramCtx : paramsCtx->funcFParam()) {
            // 解析参数类型
            SymType paramType;
            paramType.base_type = paramCtx->bType()->getText();
            paramType.is_const = false;

            // 处理数组维度
            std::vector<int> array_dims;
            if (paramCtx->getText().find("[") != std::string::npos) {
                array_dims.push_back(-1); // -1表示可变长度数组
            }

            for (auto intconstctx : paramCtx->IntConst()) {
                array_dims.push_back(std::stoi(intconstctx->getText()));
            }

            paramType.is_array = !array_dims.empty();
            paramType.array_dims = array_dims;

            // 保存参数信息
            std::string paramName = paramCtx->Ident()->getText();
```

Figure 16: visitFuncDef 函数处理过程

除了上述各项操作外，我们还实现了错误处理，对可能出现的报错进行了对应的合法处理：

```

std::any Analysis::visitErrorNode(tree::ErrorNode* node) {
    std::cerr << "Error: Invalid syntax at line "
    |         << node->getSymbol()->getLine()
    |         << ":" << node->getSymbol()->getCharPositionInLine()
    |         << " - " << node->getText() << std::endl;

    // 生成错误处理IR
    auto& builder = irGen->getBuilder();

    // 1. 生成错误信息字符串
    llvm::Value* errorMsg = builder.CreateGlobalStringPtr(
        "Syntax error encountered during compilation"
    );

    // 3. 生成程序终止指令
    builder.CreateUnreachable();

    // 验证模块以确保IR一致性
    if (verifyModule(*irGen->getModule(), &llvm::errs())) {
        std::cerr << "Generated invalid IR after syntax error" << std::endl;
    }

    exit(1);
    return nullptr;
}

```

Figure 17: 错误处理实现代码片段

上述过程里用到的各种结构体定义等封装在 analysis.h 中：


```

17 class Analysis : public CACTVisitor {
18 private:
19     // IR生成器
20     std::unique_ptr<llvm::LLVMContext> context;
21     std::unique_ptr<IRGenerator> irGen;
22
23     // 符号表相关
24     SymbolTable* globalTable;
25     SymbolTable* current;
26
27     // 类型系统
28     SymType Bool;
29
30     // 函数上下文管理
31     std::stack<std::string> current_funname;
32     std::stack<FunType> current_retype;
33
34     // 控制流状态
35     int inLoop;
36     int return_exist;
37
38     // 初始化内置函数
39     void initBuiltinFunctions();
40
41     // 多维数组初始化检查
42     bool checkMultiDimInit(const SymType& arrayType,
43                             CACTParser::ConstInitValContext* ctx,
44                             size_t dimIndex);
45
46     void generateNestedArrayInit(
47         const SymType& arrayType,
48         CACTParser::ConstInitValContext* ctx,
49         std::vector<llvm::Constant*>& initValues,
50         size_t dimIndex
51     );

```

Figure 18: analysis.h 中相关结构定义封装

综上，我们在第二部分实现的代码中，主要包括了：作用域的管理，函数信息记录与类型匹配，函数信息与变量信息合法性检查，并通过一系列 visitX 形式语法函数实现了类型检查，LLVM IR 代码内容信息收集，最终借由 IRGenerator IR 生成器完成了中间代码的生成。

Part3: 总结

我们的实验代码大致可分为上面所说的两部分，整体思路大致按照上下层的报告进行代码的完成。其中，尤其注意到词法与语法单独定义的配套存储处理，尤其是对函数或变量在语法后面出现，整体反做处理工位不宜遗漏，后面二者，代码基本考虑到了多种情况的处理，保证 LLVM IR 代码的完整生成。